Deriving Verified Abstract Machines

From Big-Step Semantics in Agda

Mahmoud M. Hamido

Department of Computer Science
RPTU Kaiserslautern-Landau

May 2026

RPTU Rheinland-Pfälzische Technische Universität Kaiserslautern Landau

Outline

1. Motivation
2. Background
3. Arithmetic
4. Exceptions
5. Simply Typed Lambda Calculus
6. Related Work & Conclusion

The Problem: Specs vs. Implementations

- Language specifications are written documents (C++ Standard, JLS, WebAssembly spec)
- Conformance is checked by *testing*

- **Flaw**: testing proves no *absence* of bugs
- A test suite is finite; the specification is universal

The Solution: Formal Verification

- **Formal verification** proves correctness for *all* inputs simultaneously
- Tool: **Agda** a dependently typed proof assistant

- CurryHoward correspondence: propositions $\leftrightarrow$ types, proofs $\leftrightarrow$ values
- The type checker *is* the proof checker

- **Central goal:** derive an abstract machine directly from big-step semantics, with machine-checked correctness and completeness

Three Case Studies

1. **Arithmetic** natural numbers, booleans, addition, conditionals
2. **Exceptions** arithmetic extended with `throw` and `try-catch`
3. **STLC** Simply Typed Lambda Calculus with λ-abstractions

For each: define four forms of semantics and prove the chain of equivalences

$$\text{Denotational} \longleftrightarrow \text{Big-Step} \longleftrightarrow \text{Machine} \longleftrightarrow \text{Small-Step}$$

Agda: Dependent Types

Types may depend on values. Classic example: length-indexed lists.

```
data Vec (A : Set) :   Set where
  []  : Vec A zero
  __ : A  Vec A n  Vec A (suc n)
```

The length lives in the *type*, not just in memory.

Termination is checked structurally. Every recursive call must be on a structurally smaller argument:

```
n+0n : (n : )  n + zero  n
n+0n zero    = refl
n+0n (suc n) rewrite n+0n n = refl
```

Agda: Propositions as Types

The empty type has no proof (is unprovable):

```
data  : Set where
-- no constructors

ex-falso : {P : Set}    P
ex-falso ()
```

Propositional equality:

```
data __ {A : Set} (x : A)
    : A  Set where
  refl : x  x
```

> `refl` is the only proof of `x  x`

> Pattern matching on `refl` unifies `x` and `y` in `x  y`

Intrinsic Typing

Instead of defining a type checker separately, index the syntax by its type. Every constructible term is *well-typed by construction*.

```
-- Value domain: maps types to Agda types
_ : Type  Set
 Nat  =
 Bool =

-- Expression type: indexed by Type
data _ : Type  Set where
    `_            :  T   T
    __            :  Nat   Nat   Nat
    test          :  Nat   Bool
    if_then_else_ :  Bool   T   T   T
```

Ill-typed programs are *not representable*.

Four Forms of Semantics

Semantics	Notation	Style
Big-Step	$e \Downarrow v$	Inductive relation; subderivations
Small-Step	$e \longrightarrow e'$	Step-by-step reduction
Denotational	$[\![e]\!]$	Recursive interpreter
Machine	$s \longrightarrow s'$	Stack + frames

$$\text{Denotational} \longleftrightarrow \text{Big-Step} \longleftrightarrow \text{Machine} \longleftrightarrow \text{Small-Step}$$

Each link proved independently; transitivity gives any pair.

Deriving Abstract Machines: Key Idea

An interpreter evaluates subexpressions *recursively*, suspending the parent until the result is available.

Make these *suspended computations* explicit as **frames**:

- A frame records surrounding context with a *hole* $\circ$ for the pending value
- Evaluating $e_1 \oplus e_2$: push frame $(\circ \oplus e_2)$, evaluate e_1
- Once v_1 is known: replace frame with $(v_1 \oplus \circ)$, evaluate e_2
- Once v_2 is known: pop frame, return $v_1 + v_2$

The collection of frames forms a **stack**. The machine halts when it reaches a return state with an empty stack.

Completeness and Correctness

Two properties must be proved for the machine to be consistent with big-step semantics:

Completeness

If $e \Downarrow v$, then for any stack stk,

$$(stk \uparrow e) \longrightarrow^* (stk \downarrow v)$$

Proved by induction on the big-step derivation, generalising over an arbitrary stack.

Correctness

If $([] \uparrow e) \longrightarrow^* ([] \downarrow v)$, then $e \Downarrow v$.
Proof: use *totality* to get v' with $e \Downarrow v'$; use completeness to get a run to $\downarrow v'$; apply *confluence* to get $v = v'$.

Arithmetic: Syntax and Types

```
data Type : Set where
    Nat : Type
    Bool : Type

_ : Type  Set
 Nat  =
 Bool =

data _ : Type  Set where
    `_              :  T   T
    __             :  Nat   Nat   Nat
    test            :  Nat   Bool
    if_then_else_   :  Bool   T   T   T
```

```
test' 0 = true; test' (suc n) = false.
```

Arithmetic: Big-Step Semantics

```
data __ :  T   T  Set where
    `_ : (v :  T)  (` v)  v
    __ : e  n  e  n
         (e  e)  (n + n)
    test : e  n
         (test e)  (test' n)
    if-then : e  true   e  v  (if e then e else e)  v
    if-else : e  false  e  v  (if e then e else e)  v
```

> **Deterministic:** `deterministic :  e  v  e  v  v  v`

> **Total:** `total :  (e :  T)  ( v  e  v)`

Arithmetic: Denotational Semantics

```
_ : {T} T T
` v = v
e e = e + e
test e = test' e
if e then e else e with e
... | true = e
... | false = e
```

Equivalence with big-step:

```
simulate : e v e v
simulate : (e : T) e v e v
```

Arithmetic: Machine Frames and Stack

```
data Hole : Set where  : Hole

data Frame : Type  Type  Set where
    __  :  T   T  Frame T Bool
    __    : Hole   Nat  Frame Nat Nat
    __    :  Nat  Hole  Frame Nat Nat
    test    : Hole  Frame Bool Nat

data Stack (Answer : Type) : Type  Set where
    []   : Stack Answer Answer
    __  : Stack Answer T  Frame T T  Stack Answer T

data State (Answer : Type) : Set where
    __ : Stack Answer T   T  State Answer
    __ : Stack Answer T   T  State Answer
```

Arithmetic: Machine Transitions

```
data __ : State Answer  State Answer  Set where
  step-`     :  stack  ` v   stack  v
  step-      :  stack  (e  e)   stack  (  e)  e
  step-      :  stack  (  e)  v   stack  (v  )  e
  step-      :  stack  (v  )  v   stack  (v + v)
  step-      :  stack  (if e then e else e)
                  (stack  (e  e))  e
  step- :  (stack  (e  e))  true    stack  e
  step- :  (stack  (e  e))  false   stack  e
```

Each transition corresponds directly to a case in the big-step semantics.

Arithmetic: Completeness

Proof by induction on the big-step derivation must generalise over an arbitrary stack:

```
complete : (e : T) (v : T) (stack : Stack Answer T)
    e v
    ( stack e) ( stack v)
```

Example case for conditionals:

```
complete (if e then e else e) v stack (if-then p p) =
    step step-
    (-transitive (complete e _ (stack (e e)) p)
    (step step-
    (complete e v stack p)))
```

Arithmetic: Correctness via Confluence

```
-confluence : e  e  e  e
   ( {e'}  e  e'  )
   ( {e'}  e  e'  )
   e  e
```

Correctness proof strategy:

1. Use **totality**: obtain v' with $e \Downarrow v'$
2. Use **completeness**: get run $([] \uparrow e) \longrightarrow^* ([] \downarrow v')$
3. Use **confluence**: the run to $\downarrow v$ and the run to $\downarrow v'$ both start from $([] \uparrow e)$ and both end in values $\Rightarrow v = v'$

Exceptions: Language Extension

```
data Exception : Set where
    err : Exception

data _ where
    ...
    throw      : Exception   T
    try_catch_ :  T  (Exception   T)   T
```

Evaluation may now produce a value *or* raise an exception:

```
data Result (T : Type) : Set where
    return  :  T  Result T
    raise   : Exception  Result T
```

Aliases: $e \Downarrow v := e \Downarrow^r \texttt{return}\ v$; $e \Uparrow ex := e \Downarrow^r \texttt{raise}\ ex$

Exceptions: Big-Step Semantics

```
data _⇓_ : ∀ {T} → Result T → Set where
  `_ : ∀ (v : T) → (` v) ⇓ return v

  _⟨_⟩_ : ∀ {e n e′ n′} → (e ⇓ e′) → (n + n′)
  -exn : ∀ {e ex} → (e ⇓ e′) → ex
  -exn : ∀ {e n e′ ex} → (e ⇓ e′) → ex

  throw-rule : ∀ {ex} → throw ex ⇓ raise ex

  try-return : ∀ {e v h} → (try e catch h) ⇓ v
  try-catch  : ∀ {e ex h r} → (try e catch h) ⇓ r
```

Every construct has a separate rule for each outcome (value or exception).

Exceptions: Machine Three Modes

The machine now operates in three modes:

```
data State (Answer : Type) : Set where
    __ : Stack Answer T   T       State Answer
    __ : Stack Answer T   T       State Answer
    __ : Stack Answer T  Exception  State Answer
```

> ↑ **eval** mode: evaluating an expression

> ↓ **return** mode: returning a value

> **unwind** mode: propagating an exception up the stack

New frame type:

```
catch : (Exception   T)  Frame T T
```

Exceptions: New Transitions

```
step-throw     :  stack  throw ex   stack  ex
step-try       :  stk  (try e catch h)
                   stk  catch h  e
step-catch-ok  :  stk  catch h  v   stk  v
step-catch-exn :  stack  catch h  ex   stack  h ex
```

Exceptions: Per-Frame Drop Rules

When unwinding, each non-catch frame must be explicitly dropped. A generic rule would lose track of which big-step constructor applies.

```
step-drop- : stack (e e) ex stack ex
step-drop- : stack ( e) ex stack ex
step-drop- : stack (v ) ex stack ex
step-drop-test : stack test ex stack ex
```

Each rule corresponds to a specific $\Uparrow$-constructor in the big-step relation (e.g. `-exn`, `-exn`).

Exceptions: Mutual Recursion in Completeness

Completeness splits into two mutually recursive lemmas:

```
complete     : e  v   ( stk  e)  ( stk  v)
complete-exn : e  ex  ( stk  e)  ( stk  ex)
```

Why mutual?

> The `try-catch` case of `complete` needs `complete-exn` for the body

> The handler case of `complete-exn` calls `complete` for the handler

Correctness extended: two terminal states ($\downarrow$ vs). If the big-step gives raise but the run ends in $\downarrow$, we get an equality between distinct constructors discharged by `ex-falso`.

STLC: Syntax with De Bruijn Indices

```
data __ : Type  Context  Set where
  Z : T  ( , T)
  S : T    T  ( , U)

data __ ( : Context) : Type  Set where
  Nat            :     Nat
  Bool           :     Bool
  __            :   Nat    Nat    Nat
  if_then_else_ :   Bool    T    T    T
  Var            : T      T
  _              : ( , T)  T    (T  T)
  _ů_            :   (T  T)    T    T
```

Identity: `(Var Z)`. Constant: `(Var (S Z))`.

STLC: Closures

Values for function types are *closures*: a saved environment paired with the body.

```
record Closure (A B : Type) : Set where
  inductive
  constructor _,_
  field
    {} : Context
       : Environment
    e  : ( , A)  B

_ : Type  Set
Nat     =
Bool    =
(A  B) = Closure A B
```

The `inductive` keyword is required: `Closure` and `_` are *mutually recursive*.

STLC: Big-Step Semantics

```
data ___ : Environment     T   T  Set where
  ...
  VAR  :   Var x  lookup  x
  ABS  :    e    , e
  app  :   e    , e
          e  v
        ( , v)  e  v
          e ů e  v
```

Totality requires a `TERMINATING` pragma the recursive call on the closure body is not structurally smaller:

```
{-# TERMINATING #-}
total :  (e :   T)   ( v    e  v)
```

STLC: Denotational Semantics Two Versions

Direct (function types as Agda functions):

```
_ : Type  Set
 A  B  =  A    B

interpret  ( e) =
     z  interpret ( , z) e
interpret  (e ů e) =
    (interpret  e) (interpret  e)
```

Closure (explicit closures):

```
evaluate  ( e) =   , e
evaluate  (e ů e) =
    apply (evaluate  e)
          (evaluate  e)

apply   , e  v =
    evaluate ( , v) e
```

Related by **logical relations**: $f_1 \sim_{\langle T \Rightarrow U \rangle} f_2$ iff for all related arguments, results are related.

STLC: Machine Three Application Frames

The environment is carried *with the state*. Application uses three successive frames:

```
data Frame : Type  Type  Set where
  app : Hole    A        Frame B (A  B)    -- evaluate e
  app :  (A  B)  Hole   Frame B A           -- evaluate e
  app : Environment    (A  B)   A
        Hole  Frame B B                          -- evaluate body
```

1. app: evaluate e_1, obtain closure
2. app: evaluate e_2, obtain argument v_2
3. app: switch to closure's env δ^c, evaluate body e^c; restore caller's env on return

STLC: Machine Transitions for Application

```
step-var : ( stack Var x)
           ( stack lookup x)
step-    : ( stack ( e))
           ( stack   , e )
step-ů   : ( stack e ů e)
           ( stack app e e)
step-ů   : ( stack app e v)
           ( stack app v e)
step-ů   : ( stack app   , e   v)
           (( , v) stack app   , e v   e)
step-ů   : ( stack app f v   v)
           ( stack v)
```

STLC: Open Problem Small-Step Equivalence

Small-step performs *substitution*:

$$(\lambda\, e) \cdot v \longrightarrow e[v/0]$$

Big-step uses *environment lookup*: $(\delta^c, v_2) \vdash e^c \Downarrow v$

To state equivalence we need `embed : T T`:

```
embed {T = T  T}  , e  =  ?   -- body is in , not ,T
```

The captured environment δ^c is defined in context $\Gamma^c \neq \emptyset$ it *cannot* be embedded. This equivalence remains an **open problem**.

Related Work

Hutton (2002) Evaluator → CPS → defunctionalization → machine. A frame here ≡ defunctionalized continuation. Correctness by construction.

Hutton & Wright (2004) Same approach extended with exceptions. Continuation type modified to account for exception-raising.

Hutton et al. (2006) Verified compiler to a stack-based machine; handlers pushed as code; exceptions unwind to find a handler.

Hutton et al. (Agda) Agda formalization of the above without explicit correctness proofs.

This thesis Derives machines from the *structure of big-step derivations* directly. Machine-checked correctness and completeness.

Summary of Results

	Arithmetic	Exceptions	STLC
Big-Step $\leftrightarrow$ Denotational	✓	✓	✓
Big-Step $\leftrightarrow$ Machine	✓	✓	✓
Big-Step $\leftrightarrow$ Small-Step	✓	✓	open

- Uniform methodology: completeness by induction on big-step; correctness by totality + completeness + confluence
- Chain of equivalences means any pair of semantics can be related

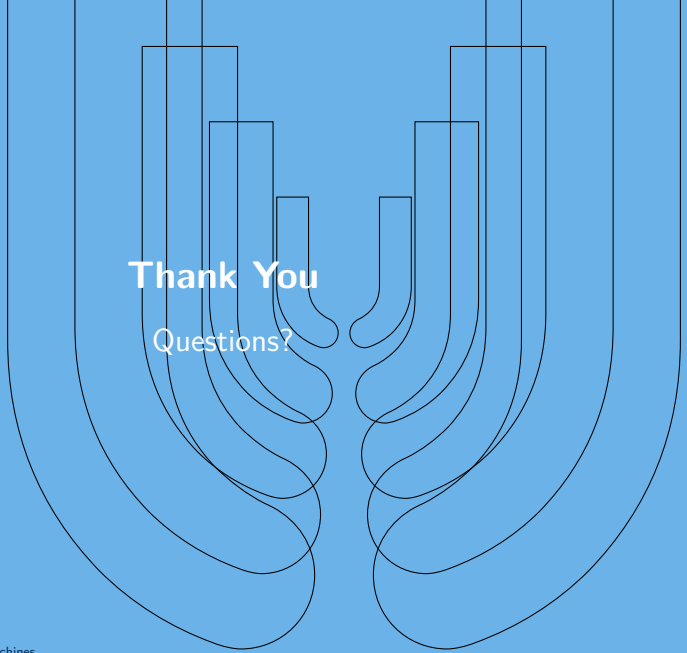
Limitations & Future Work

Limitations

1. **Totality assumption**: big-step cannot represent non-terminating programs
2. **TERMINATING pragma** in STLC totality: accepted on trust, not verified structurally
3. **STLC small-step $\leftrightarrow$ big-step**: open due to substitution vs. environment mismatch

Future Work

- Coinductive big-step or small-step as the primary semantics
- Structural totality for STLC (eliminate pragma)
- Kripke logical relations for STLC small-step equivalence
- Abstract the methodology into a reusable framework



Thank You

Questions?